

```
> (defmacro setq-literal (place literal)
  `(setq, place ', literal))
SETQ-LITERAL

>(setq-literal a b)
B

> a
B

> (defmacro reverse-cons (rest first)
  `(cons, first, rest))
REVERSE-CONS

>(reverse-cons nil a)
(B)
```

setq-literal works like *setq*, except that neither argument is evaluated. Remember that *setq* evaluates its second argument. The body of *setq-literal* has a form that begins with a ``` (back-quote). A back-quote behaves like *quote*—suppressing evaluation of all the enclosed forms—except where a comma appears within the back-quoted form. A symbol following the comma is evaluated.

So, in our call to *(setq-literal a b)* above, this is what needs to be done:

1. Bind *place* to the symbol *a*.
2. Bind *literal* to the symbol *b*.
3. Evaluate the body ``(setq, place ', literal)`, following these steps:

- Evaluate *place* to get the symbol *a*.
- Evaluate *literal* to get the symbol *b*.
- Return the form *(setq a 'b)*.

Neither the back-quote nor the commas appear in the returned form. Neither *a* nor *b* is evaluated in a call to *setq-literal*, but for different reasons; *a* is unevaluated because it appears as the first argument of *setq*; *b* is unevaluated because it appears after a quote in the form returned by the macro. The operation of *(reverse-cons nil a)* is similar:

1. Bind *rest* to the symbol *nil*.
2. Bind *first* to the symbol *a*.
3. Evaluate the body ``(cons, first, rest)`, following these steps:
 - Evaluate *first* to get the symbol *a*.
 - Evaluate *rest* to get the symbol *nil*.
 - Return the form *(cons a nil)*.

Both arguments of *reverse-cons* are evaluated because *cons* evaluates its arguments, and our macro body doesn't quote either argument. *a* evaluates to the symbol *b*, and *nil* evaluates to itself.

Macro expansion

Lisp evaluates the code produced by the macro immediately after producing it. Therefore, when you use a macro, you

get to see only the result of the entire evaluation of the macro, but you do not get to see the code produced by the macro along the way. Since it is sometimes useful in debugging to view this intermediate code, Lisp provides us with a way of doing so. The function *macroexpand* takes as input an s-expression that may be a call to a macro. If it is, *macroexpand* will apply the macro to its arguments to produce the object code. It will not evaluate this code, but merely return it. The Lisp interpreter uses *macroexpand* to expand a call to a macro that it is trying to evaluate. Using this function will give you an accurate picture of what the Lisp interpreter itself sees. Here's how it works for the example given above:

```
> (macroexpand '(setq-literal a b))
(SETQ A (QUOTE B))
T

> (macroexpand '(reverse-cons nil a))
(CONS A NIL)
T
```

Since *macroexpand* is a function, it evaluates its arguments. This is why you have to quote the form you want expanded.

A convenient macro

It is generally appropriate to enclose function arguments inside a call to the special function.

In particular, the expression *(function (lambda...))* occurs frequently. Common Lisp allows the shorthand `#'x` to stand for *(function x)*, just as `'x` stands for *(quote x)*. Let's write a special macro to help abbreviate this particular combination. We can define a macro *flambda* that expands into *(function (lambda...))* as follows:

```
> (defmacro flambda (&rest l)
  (list 'function (cons 'lambda l)))
FLAMBDA

> (macroexpand '(flambda (x y) (cons y x)))
(FUNCTION (LAMBDA (X Y) (CONS Y X)))
T
```

Since a lambda expression may contain any number of forms, we used the *&rest* parameter designator to define a rest parameter. This parameter will get assigned the list of all the arguments supplied to *flambda*. For example, in the call to *flambda* above, the rest parameter *l* gets assigned the value *((x y) (cons y x))*. *flambda* then just sticks the atom *lambda* on the front of this list, and then puts the resulting lambda expression in a list that begins with the symbol *function*. Thus, the sample 'flambda expression' given above as an argument to